

Sider Fashion — Security & API Validation Audit

Project: Sider Fashion full-stack storefront + admin panel (React 19 / Vite / Express / Supabase) **Date:** 9 September 2026 **Scope:** `server.ts` (3,068 lines, 72 API routes), `server/supabase.ts` , and the client service layer **Constraint applied:** fix defects **without changing application logic** — no feature was added, removed, or redesigned

1. Executive Summary

The audit found that **the entire administrative API was unauthenticated**. Every catalog, order, CMS, pricing and payment endpoint accepted anonymous writes, and two separate flaws let any visitor impersonate the Store Owner outright. Admin password hashes were being published to every visitor of the storefront home page.

27 defects were found and fixed across three categories:

Category	Count	Severity spread
Authentication & authorization	6	4 Critical, 2 High
Data exposure & hardening	11	1 Critical, 5 High, 5 Medium
API route validation & data integrity	10	2 High, 5 Medium, 3 Low

Verification: 139 API assertions, 24 browser end-to-end assertions, a 21-tab admin UI sweep, and 17 exploit probes were run. Before the fixes, **16 of the 17 exploit probes succeeded**. After, **none do**, and every functional test passes in both dev and production builds.

Ten further issues could not be fixed without changing behaviour and are documented in §6 as recommendations for you to decide on.

2. Confirmed Exploits — Before vs After

These were run as live HTTP requests against a running server, first on the original code, then on the fixed code.

#	Attack (all with no credentials)	Before	After
V1	Forge an owner session: <code>POST /api/admin/session/verify {"userId": "admin-saon"}</code>	✓ returned <code>valid:true</code> + the owner's record	✗ <code>valid:false</code>
V2a	Delete a real admin account	✓ deleted "Savar Dispatch & Factory Lead"	✗ 401
V2b	Create a backdoor admin account	✓ account created	✗ 401
V2c	Log in with the backdoor account	✓ logged in as <code>admin</code>	✗ n/a
V3	Read the full admin roster	✓ 4 accounts, incl. last-login IPs	✗ 401
V4	Steal admin PBKDF2 hashes + salts	✓ 4 hashes + salts leaked	✗ absent from payload
V5	Write/deface the product catalog	✓ product created	✗ 401
V6	Overwrite bKash/Nagad payment accounts	✓ attacker field written	✗ 401
V7	Dump all customer orders (name, phone, address)	✓ 5 orders	✗ 401
V8	Trigger a full database replace from Supabase	✓ reached the handler	✗ 401
V9	Inflate inventory with a negative order quantity	✓ stock 99 → 599	✗ 99 → 98
V10	Store a <code>javascript:</code> URL rendered as a storefront link	✓ accepted	✗ 401 + rejected
V11a	Grow the audit log past its 500-entry cap	✓ 56 → 520	✗ unchanged
V11b	Persist arbitrary attacker JSON into the database	✓ stored verbatim	✗ field-allowlisted
V12	Read API responses cross-origin from any website	✓ <code>ACAO: *</code>	✗ no ACAO header
V13	Bypass brute-force lockout via spoofed <code>X-Forwarded-For</code>	✓ 8 failures, never blocked	✗ blocked after 5
V14	Unknown <code>/api</code> route returns a non-JSON body	⚠ 404 with a non-JSON body → client <code>res.json()</code> threw	✗ proper JSON 404

V2 is the most serious. The caller lookup was `db.adminUsers.find(u => u.id === currentAdminId || u.role === 'owner')`. The `|| u.role === 'owner'` arm matched the Store Owner for *any* request, so a request carrying no credentials at all resolved to the owner and passed the "only the owner may do this" check. A stranger could delete your staff accounts and mint their own.

3. Authentication & Authorization Fixes

3.1 Privilege escalation via the owner fallback — Critical

`POST /api/admin/users`, `DELETE /api/admin/users/:id`

The `|| u.role === 'owner'` fallback was removed. The caller is now resolved from a server-side session, never from a client-supplied id.

3.2 Session forgery — Critical

`POST /api/admin/session/verify`

Was `const targetUserId = session?.userId || userId;` — with no valid token it fell back to whatever `userId` the caller sent, then returned `valid:true` plus that user's record. Now a live, unexpired server-side session token is required; there is no fallback.

3.3 Identity taken from the request body — Critical

`PUT /api/admin/users/:id`

`currentAdminId` arrived in the JSON body, so any caller could claim to be the owner by naming the owner's id (which `GET /api/admin/users` handed out for free). Identity now comes from the session.

3.4 No authentication on any mutating endpoint — Critical

Two guards were introduced, `requireAdmin()` and `requireOwner()`, and applied to **45 route handlers**. The token is read from `Authorization: Bearer`, `X-Admin-Token`, or the legacy body/query `token` field.

Now protected (admin session required): products · categories · sizes · colors · contacts · social-links · hero-slides · faqs · homepage-sections · settings · policies · payment-accounts · coupons · campaigns (writes) · orders (read/update/delete) · wholesale (read/update/delete) · audit-logs · analytics events (read) · admin users (all) · supabase sync-now / pull-now · all three AI endpoints

Deliberately left public (customer-facing, unchanged): `GET /api/health` · `GET /api/sync` · `GET /api/sync/events` · all storefront `GET` reads · `POST /api/orders` · `POST /api/wholesale` · `POST /api/analytics/events` · login / logout / session-verify / forgot-password

On the client, a new `src/services/authHeaders.ts` reads the token the login flow already stores in `localStorage` and attaches it. 50 call sites across `adminStoreService.ts`, `orderService.ts`, `AdminAIDashboard.tsx` and `lib/supabase.ts` were updated. No call signature changed.

3.5 Hardcoded passcode backdoor — High

```
const legacyPasscodes = ['2026','sider2026','admin123','sideradmin','773063','111222','Sider@2026'];
const isLegacyPassValid = !user.passwordHash && legacyPasscodes.includes(inputPassword);
```

Any account reaching login without a stored hash could be signed into with a publicly guessable string. Removed. No legitimate access is lost: accounts without a hash are given one during database load.

3.6 Brute-force logout bypass — High

The client IP came from `req.headers['x-forwarded-for']`, which the attacker controls, and the rate-limit key was built from it. Rotating the header value gave every attempt a fresh bucket, so the 5-attempt logout never engaged. `X-Forwarded-For` is now honoured **only** when `TRUST_PROXY=true` is set; otherwise the real socket address is used.

4. Data Exposure & Hardening Fixes

Fix	Severity	Detail
Password hashes removed from <code>GET /api/sync</code>	Critical	The public sync payload included <code>adminUsers</code> complete with <code>passwordHash</code> and <code>passwordSalt</code> — delivered to every storefront visitor on page load. Nothing in the client reads this field, so projecting it out changes no behaviour.
Last-login IPs removed	High	Now visible only to the owner, or to the account holder for their own record.
Wildcard CORS replaced	High	<code>Access-Control-Allow-Origin: *</code> let any website read API responses from a victim's browser. Now an allowlist via <code>CORS_ALLOWED_ORIGINS</code> ; same-origin traffic (the app itself) is unaffected.
Reset OTP now uses a CSPRNG	High	<code>Math.floor(100000 + Math.random() * 900000)</code> — the comment claimed "cryptographically random" but <code>Math.random()</code> is predictable from observed output. Now <code>crypto.randomInt</code> .
<code>javascript:</code> / <code>data:</code> URLs rejected	High	<code>socialLinks[].url</code> and <code>settings.facebookUrl</code> are rendered as anchor <code>href</code> s in <code>Footer.tsx:281</code> . A stored <code>javascript:</code> URL was a stored-XSS vector. Both fields now require <code>http(s)</code> .
Supabase pull validated	High	<code>POST /api/supabase/pull-now</code> replaced the whole database with an unvalidated remote document. A remote doc missing <code>adminUsers</code> made every later request throw and locked all admins out permanently. Now owner-only, shape-validated, and normalized.
OTP compared in constant time	Medium	Prevents timing-based code recovery.
Session & rate-limit maps purged	Medium	<code>activeAdminSessions</code> , <code>passwordResetCodes</code> and <code>failedLoginAttempts</code> grew without bound. Now swept every 10 minutes.
Security headers added	Medium	<code>X-Content-Type-Options: nosniff</code> , <code>X-Frame-Options: SAMEORIGIN</code> , <code>Referrer-Policy</code> , <code>X-Permitted-Cross-Domain-Policies</code> , <code>Cache-Control: no-store</code> on <code>/api/*</code> , and <code>x-powered-by</code> disabled.
Audit log capped & allowlisted	Medium	<code>POST /api/audit-logs</code> stored arbitrary client JSON verbatim and — unlike the internal <code>logAction</code> helper — applied no 500-entry cap , so repeated posts grew <code>store.json</code> without limit (verified: 56 → 520 entries). Now admin-only, field-allowlisted, length-capped, and capped at 500.
Analytics events allowlisted	Medium	<code>POST /api/analytics/events</code> is necessarily public; it now stores a pinned set of length-capped fields instead of whatever JSON arrives.

5. API Route Validation & Data Integrity Fixes

5.1 Negative order quantity inflated stock — High

```
p.stock = Math.max(0, (p.stock || 0) - (Number(item.quantity) || 1));
```

`Math.max(0, ...)` only guards the lower bound. With `quantity: -500`, `stock - (-500)` adds 500 units. Verified: a single crafted order took a product from 99 to 599 units. Quantity is now clamped to a positive integer, with the same default of 1 for legitimate input.

5.2 Order & inquiry ID collisions — High

```
const padNum = (db.orders.length + 1).toString().padStart(6, '0');
order.orderId = `SF-2026-${padNum}`;
```

IDs derived from array length are reused as soon as any record is deleted. `PUT /api/orders/:id` and `DELETE /api/orders/:id` both use `findIndex(o => o.orderId === id)`, which matches only the **first** record — so a status update or deletion aimed at one customer silently hits a different customer's order.

This is not hypothetical: your committed `data/store.json` already contains `SF-2026-000003` twice (customers "WebMarkio" and "Robiul"). It is also the source of the React *"Encountered two children with the same key"* warning on the Order Processing tab.

Fixed: IDs are now allocated from the highest existing sequence number and checked for uniqueness; a colliding client-supplied ID is re-issued rather than stored as a duplicate. The same fix was applied to wholesale inquiry IDs. **The existing duplicate in your data still needs a manual correction — see §6.7.**

5.3 Non-object payloads corrupted stored records — Medium

`db.products[idx] = { ...db.products[idx], ...updated }` with `updated` as an array spread numeric keys into the record; with `updated` undefined it wrote nothing yet still returned `success: true`. Every PUT/POST route now rejects non-object payloads with a 400.

5.4 Silent write failure — Medium

`PUT /api/homepage-sections` discarded any non-array payload and still returned `success: true` — the admin UI showed "saved" for a write that never happened. It now returns 400.

5.5 Unhandled 500s from non-string input — Medium

Nine routes called `.trim()`, `.toLowerCase()` or `.replace()` directly on request fields. `POST /api/sizes {"size": 12345}` returned a 500; so did `product.code`, `color.name`, `category.key`, `faq.question`, `slide.title`, `campaign.campaignName`, the AI chat `message`, and any admin record with a missing `email`. All are now type-checked.

5.6 Password length check passed for arrays — Medium

`password.length < 6` is satisfied by `[1,2,3,4,5,6]`, which then threw inside `crypto.pbkdf2Sync`. All password fields now require an actual string (6–200 chars).

5.7 Weak email validation — Low

`cleanEmail.includes('@')` accepted `"@"`. Replaced with a structural check.

5.8 Error handling — Low (three defects)

Malformed JSON returned an HTML stack trace; unknown `/api/*` paths returned a non-JSON body that made the client's `res.json()` throw (this is why the orphaned `/api/admin/change-password` client call surfaced as a

generic "connection error"). Added: a JSON 400 for malformed bodies, a JSON 404 for unknown API paths, a 413 for oversized payloads, and a catch-all handler that logs server-side and returns a generic message without leaking stack traces.

6. Not Fixed — Recommendations Requiring Your Decision

Each of these would change behaviour, so per the "no logic changes" constraint they are reported rather than applied.

6.1 GET /api/sync still exposes customer PII — HIGH, please read. The public sync payload still contains every order (customer name, phone, full address), the audit log, and marketing campaign figures. Credentials have been stripped, but the PII remains. It cannot simply be removed because both the cross-device order-tracking feature and the admin dashboard read orders from this payload. *Suggested fix:* gate

`orders / wholesaleInquiries / auditLogs / campaigns / events` in `/api/sync` behind the admin session, and add a narrow public `POST /api/orders/lookup` that requires order ID **and** phone and returns only matching orders. `OrderService.lookupOrders` and `OrderTrackingModal` would become async. I can implement this on request.

6.2 Order totals are client-supplied — HIGH. `POST /api/orders` stores `order.total`, `subtotal` and `discount` exactly as sent. A customer can order at any price they choose. A fix means recomputing totals server-side from the catalog, coupons and delivery rules — real new logic.

6.3 Default passwords are still live — HIGH. Three of four accounts still use the seeded defaults, which are visible in the source: `abirhosensaon@gmail.com` and `siderfashion.bd@gmail.com` use `Sider@2026`; `factory@siderfashion.com` uses `Admin@2026`. **Change these now.**

6.4 Supabase anon key is committed — HIGH. The key is hardcoded in `src/lib/supabase.ts`, `server/supabase.ts` and `.env.example`. Anon keys are public by design, but your schema grants `CREATE POLICY ... FOR ALL USING (true) WITH CHECK (true)` to the anon role, making it a full read/write credential for the store database. Restrict the RLS policies and rotate the key.

6.5 OTP printed to server logs. When SMTP is unconfigured the reset code is logged in plaintext. Configure `SMTP_HOST / SMTP_USER / SMTP_PASS`.

6.6 PBKDF2 at 10,000 iterations. Below current guidance (~210,000 for HMAC-SHA512). Raising it invalidates every stored hash, so it needs a rehash-on-next-login migration.

6.7 Repair the duplicate order ID. `SF-2026-000003` appears twice in `data/store.json`. Until one is renumbered, admin updates to that ID will hit the wrong customer's order.

6.8 Orphaned client method. `AdminStoreService.changePassword()` calls `/api/admin/change-password`, which does not exist server-side. Nothing in the UI calls it. I did **not** add the route, because an unauthenticated password-change endpoint would be a fresh vulnerability. Either delete the dead method or specify an authenticated design.

6.9 Sessions are in-memory. A server restart signs all admins out. Acceptable for a single-instance deployment; a multi-instance deployment needs shared session storage.

6.10 No rate limit on order creation. `POST /api/orders` is public and unthrottled — open to spam order floods.

7. Verification

All tests were executed against a running server; nothing was assumed.

Suite	Assertions	Result
API security & validation (<code>apitest.mjs</code>)	139	139 passed, 0 failed
Exploit probes, original build	17	16 succeeded → vulnerabilities confirmed real
Exploit probes, fixed build	17	0 succeeded
Chromium end-to-end (<code>e2e.mjs</code>)	24	24 passed, 0 failed
Admin UI tab sweep (21 tabs)	21	0 API errors, 0 blank panels
TypeScript <code>tsc --noEmit</code>	—	clean
<code>npm run build</code> + production server	—	clean; SPA routing, static assets and all guards verified under <code>NODE_ENV=production</code>

Browser coverage (Chrome 150, headless): storefront loads for an anonymous visitor with products rendering; anonymous checkout and wholesale inquiry succeed; admin login page gates the panel; owner login issues and stores a session token; the dashboard renders with live data (₹21,040 revenue, 6 orders, 1,039 units); all 21 admin tabs open without error; an authenticated product create/delete round-trip succeeds; logout revokes the session server-side and re-locks the panel.

Two regressions I introduced were caught by the browser sweep and fixed:

- `fetchAdminUsers()` used a bare `fetch(url)` with no headers, so the Admin & Staff Accounts tab got a 401. Fixed.
- `logoutAdminAsync()` wrote its audit entry *after* the logout request had already revoked the token, so the entry was rejected and lost. The audit call now runs first.

Test data hygiene: `data/store.json` was backed up before testing and restored afterwards; `git diff` confirms it is byte-identical to the committed baseline.

8. Files Changed

File	Change
<code>server.ts</code>	All server-side fixes: auth guards, validation helpers, security headers, error handling, ID allocation
<code>src/services/authHeaders.ts</code>	New. Shared session-token accessor (standalone, to avoid a circular import)
<code>src/services/adminStoreService.ts</code>	Attach the session token to admin calls; fix <code>fetchAdminUsers</code> ; reorder logout audit
<code>src/services/orderService.ts</code>	Attach the token to admin order/wholesale calls; public checkout left untouched
<code>src/components/admin/AdminAIDashboard.tsx</code>	Attach the token to the three AI endpoints
<code>src/lib/supabase.ts</code>	Attach the token to the manual cloud sync trigger

Net: **+894 / -264 lines**, no dependencies added.

New environment variables (all optional, safe defaults)

Variable	Default	Purpose
<code>TRUST_PROXY</code>	<code>false</code>	Set to <code>true</code> only behind a reverse proxy you control, so <code>X-Forwarded-For</code> is honoured for rate limiting
<code>CORS_ALLOWED_ORIGINS</code>	empty	Comma-separated origins allowed to call the API cross-site. Leave empty unless a separate front-end domain needs access

9. Suggested Next Steps

1. **Change the three default admin passwords today** (§6.3).
2. **Tighten the Supabase RLS policies and rotate the anon key** (§6.4).
3. Decide on §6.1 (customer PII in `/api/sync`) — the highest remaining risk. I can implement the lookup-endpoint approach.
4. Decide on §6.2 (server-side order total recalculation).
5. Renumber the duplicate order ID in `data/store.json` (§6.7).
6. Configure SMTP so reset codes stop being written to logs (§6.5).